



SAPMS: A Semantic-Aware Privacy-Preserving Multi-keyword Search Scheme in Cloud

Qian Zhou¹, Hua Dai^{1,2(✉)}, Zheng Hu¹, Yuanlong Liu¹, and Geng Yang^{1,2}

¹ Nanjing University of Posts and Telecommunications, Nanjing 210023, China
daihua@njupt.edu.cn

² Jiangsu Security and Intelligent Processing Lab of Big Data,
Nanjing 210023, China

Abstract. Most of the traditional privacy-preserving search schemes adopt TF-IDF model which is on the basis of keyword frequency statistics. The embedding semantic association between keywords and documents are not considered. To solve this problem, we propose an efficient semantic-aware privacy-preserving multi-keyword search scheme over encrypted cloud data. The LDA topic model is adopted to generate the topic information-embedded vectors for documents and queried keywords. The homomorphic encryption on vectors is used to perform privacy-preserving semantic relevance score computation between queried keywords and documents. To achieve efficient search processing, a novel tree-based index is designed, which is constructed following the divisive hierarchical clustering algorithm. By using the tree-based index, a depth-first privacy-preserving multi-keyword search algorithm is proposed. The experimental results show that the proposed scheme outperforms the existing schemes in terms of the semantic precision of search results and the search time cost.

Keywords: Cloud computing · Searchable encryption · Multi-keywords search · Semantic-aware · Privacy-preserving

1 Introduction

Nowadays, cloud computing is becoming more and more popular and useful for enterprises, which provides dynamic, easy-scalable and virtualized resources through the Internet. Data and software of enterprises could be outsourced and deployed in the cloud for cost saving. However, the outsourcing of data makes the data owner losing the independent and exclusive control on his/her data, which could lead privacy leakage issues. The cloud service provider could obtain the valuable information of the outsourced data by statistical interfering, etc. To protect data privacy, an easy idea is to encrypt the data before it is outsourced to the cloud. But the encrypted data is difficult to search and use, which severely decreases the availability of the data. It is a challenge to provide searchability

and privacy preservation on the outsourced data, simultaneously. Therefore, the research on searchable encryption schemes is a necessary.

Recently, many searchable encryption schemes have been proposed, such as single keyword search schemes [3, 19, 22, 24, 27, 28, 32, 33] and multi-keyword search schemes [2, 4–9, 11–13, 15–18, 20, 21, 25, 26, 30, 31]. Most of these schemes adopt the term frequency-inverse document frequency model (TF-IDF) based vector space model to generate vectors to represent documents and searched keywords, and these vectors are encrypted by the random invertible matrix based vector encryption method [2] for privacy preservation. The encrypted vectors are organized into the sophisticate encrypted indexes which support the privacy-preserving multi-keyword search processing over the encrypted cloud data. However, the TF-IDF model is a statistical model which utilizes the quantized frequency statistics to evaluate the importance of keywords to a set of documents. It ignores the semantic association between keywords and documents, which could lead to unsatisfied results for users. At the same time, due to the large number of keywords in the dictionary extracted from the whole documents, the generated vectors following the TF-IDF model could have an extremely high dimension and sparsity. It costs a lot of storage space and search time. The previous work in [5] firstly proposed a LDA-based semantic-aware search scheme LDA-EMRSE. Documents are trained by using the LDA model and represented by lower dimensional topic vectors. But the semantic relation between documents are not considered when constructing the search index, there is still room of improvement in search precision and efficiency.

In this paper, we proposed a semantic-aware privacy-preserving multi-keyword search scheme (SAPMS). The data owner first uses the LDA model to train documents and generate the topic information-embedded vectors for documents. The vectors are taken as the input of the divisive hierarchical clustering algorithm, and a novel binary clustering tree (BCI-tree) is formed which is used as the index for search processing. The index is encrypted by using the homomorphic matrix encryption, and the encrypted index is outsourced to the cloud server together with the encrypted documents. When starting a multi-keyword search to require k highest relevant documents in semantic, the data user converts the queried keywords into the trapdoor which is submitted to the cloud server as the search command. The cloud server executes the search by using the encrypted index and returns the result to the data user. At last, the data user performs the decryption to obtain the plaintext result. The experimental results show that the proposed scheme has better performance in terms of the search result precision and search time cost.

2 Notations and Preliminaries

2.1 Notations

- F : a set of n documents, $F = \{f_1, f_2, \dots, f_n\}$, whose encrypted form is $\hat{F} = \{\hat{f}_1, \hat{f}_2, \dots, \hat{f}_n\}$.
- D : a dictionary having h keywords, $D = \{w_1, w_2, \dots, w_h\}$, which is extracted from documents of F .

- T : a set of m topics, $T = \{t_1, t_2, \dots, t_m\}$.
- Γ : An $n \times m$ -dimensional document-topic correlation matrix, whose encrypted form is $\hat{\Gamma}$.
- Ψ : an $h \times m$ -dimensional keyword-topic correlation matrix.
- P_T : an m -dimensional topic probability vector.
- P_W : an h -dimensional keyword probability vector.
- $\mathcal{I}$: a binary tree-based index, whose encrypted form is $\hat{\mathcal{I}}$.
- Q : a search with multiple queried keywords, $Q \subseteq D$.
- V_Q : an m -dimensional query topic vector, whose encrypted form is $\hat{V}_Q$.

2.2 Preliminaries

LDA Model. The Latent Dirichlet Allocation model (LDA) is an unsupervised learning algorithm proposed by David Blei [1]. By using the LDA model to train the documents of F , a document-topic correlation matrix Γ and a keyword-topic correlation matrix Ψ will be generated, which describe the numerical topic relevance between topics and documents and keywords, respectively. Each document and keyword can be represented by a document topic vector and a keyword topic vector, respectively. Given a document $f_i \in F$, a topic $t_j \in T$ and a keyword $w_e \in W$, $\Gamma[i]$ is the document topic vector of f_i , where $\Gamma[i][j]$ records the topic relevance score between f_i and t_j ; $\Psi[e]$ is the keyword topic vector of w_e , where $\Psi[e][j]$ records the topic relevance score between w_e and t_j .

Sematic Relevance Measurement: In this paper, documents and searches are represented by the vectors embedding the topic semantic information. The sematic relevance score between a document and a search is the inner product of the corresponding vectors. Assuming that $\Gamma[i]$ and V_Q are the document topic vector of f_i and the query topic vector of Q , respectively, the semantic relevance score between f_i and Q is the inner product between $\Gamma[i]$ and V_Q , i.e. $Score(f_i, Q) = \Gamma[i] \cdot V_Q$.

Divisive Hierarchical Clustering. The divisive hierarchical clustering algorithm [23] is one of the hierarchical clustering algorithms, which can divide a large cluster into multiple small sub-clusters recursively and form a binary clustering tree. Each node in the tree represents a cluster and its left and right child nodes represent the generated two sub-clusters.

Secure Inner Product Operation. The secure inner product [2] is implemented by a homomorphic matrix encryption. We assume that x and y are two given n -dimensional vectors and M is a random $n \times n$ invertible matrix. Here, M is used as a private key. The vectors x and y are encrypted by M and the encrypted vectors are denoted as $\hat{x}$ and $\hat{y}$, where $\hat{x} = M^T \cdot x$ and $\hat{y} = M^{-1} \cdot y$. The inner product of x and y is equal to the inner product of $\hat{x}$ and $\hat{y}$ because $\hat{x} \cdot \hat{y} = (M^T x) \cdot (M^{-1} y) = (M^T x)^T (M^{-1} y) = x^T M M^{-1} y = x \cdot y$. It indicates that the inner product of two vectors can be obtained without knowing the plaintext values of any dimensions of vectors.

3 Problem Statement

System Model. The system model is the shown in Fig. 1, which involves three different entities: the data owner, the data user and the cloud server. The data owner owns a collection of documents and he/she outsources the encrypted documents and index to the cloud server and shares the secure key and other information with the data user. The data user generates a trapdoor when he/she starts a multi-keyword search. The trapdoor is submitted to the cloud server as the search command. The cloud server performs the search processing by using the encrypted index and returns the search result in ciphertext to the data user. When the data user receives the encrypted result, he/she decrypts it to get the final plaintext search result.

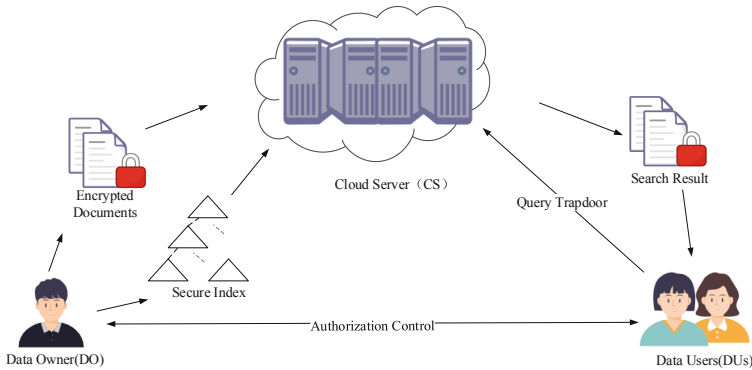


Fig. 1. An example of BCI-tree search.

Threat Model. In this paper, we adopt the same threat model as most of the existing searchable encryption schemes [2–4, 8, 9, 11–13, 17–21, 24–28, 30–32], which is the “honest-but-curious” threat model. In this model, the cloud server is assumed to execute the search processing honestly and returns the correct search results to the data user. However, the cloud server is curious about the information of the outsourced data by statistical analyzing or other methods.

Design Goals. Given a multi-keyword search Q , the proposed search scheme can achieve the following design goals.

- 1) **Semantic-aware multi-keyword search.** In the scheme, the cloud server can return the k documents which have the highest k semantic relevance scores to the queried keywords.
- 2) **Search efficiency and accuracy.** The scheme realizes efficient and accurate search by using semantic model, special index and efficient search algorithm.
- 3) **Privacy preservation.** The scheme can protect the privacy of documents, index and queried keywords.

4 The Proposed Search Scheme

There are three modules in the proposed search scheme, which are the data preprocessing module, index building module and multi-keyword search module.

4.1 Data Preprocessing Module

The data preprocessing module mainly includes the private key generation and the LDA topic model training, which are described as the algorithms *GenKey* and *LDATrain*.

$K \leftarrow \mathbf{GenKey}(\mathbf{1}^{l(n)})$. The *GenKey* algorithm is to generate a set of private keys $K = \{S, M_1, M_2, k_f\}$, where S is an m -bit random vector, M_1 and M_2 are two random $m \times m$ invertible matrices and k_f is the key used to encrypt documents. K is only shared between the data owner and the authorized data user while the cloud server has no idea of it.

$(\Gamma, \Psi, P_T, P_W) \leftarrow \mathbf{LDATrain}(F)$. The *LDATrain* algorithm is to generate the document-topic correlation matrix Γ , the keyword-topic correlation matrix Ψ , the topic probability vector P_T and the keyword probability vector P_W . The algorithm is shown as the following steps.

- 1) Taking the documents of F as the input, the data owner uses the LDA model to perform the train processing, and the document-topic correlation matrix Γ and the keyword-topic correlation matrix Ψ are generated.
- 2) The data owner generates the topic probability vector P_T . The i th dimension of P_T stores the prior belief of the topic t_i existing in F . Assuming that documents in F have the same importance, the prior probability of t_i is the average probability that each document of F belongs to t_i .

$$P_T = \frac{1}{n} \cdot \sum_{j=1}^n \Gamma[j]. \quad (1)$$

- 3) The data owner generates the keyword probability vector P_W . The i th dimension of the vector stores the prior belief of the keyword w_i existing in F .

$$P_W = P_T \cdot \Psi^T. \quad (2)$$

In the end of data preprocessing module, the data owner shares K , Ψ , P_T and P_W generated in the above steps to the data user.

4.2 Index Building Module

This section proposes the structure definition and the construction algorithm of a novel binary clustering tree based index which is denoted as the BCI-tree.

Definition 1. Maximized Filter Vector. Given a set of m -dimensional vectors $G = \{V_1, V_2, \dots, V_x\}$, the maximized filter vector of G is denoted as $MFV(G)$ which is an m -dimensional vector. The i th dimension of $MFV(G)$ stores the maximum of the value set consisting of the i th dimension of each vector in G . Assuming that $V = MFV(G)$, we have $V[i] = \max\{V_j[i] | V_j \in G\}$.

Definition 2. BCI-tree. The BCI-tree is a binary tree which is constructed by the divisive hierarchical clustering algorithm. The tree is used as the search index and denoted as $\mathcal{I}$. Each node u in $\mathcal{I}$ is a four-element tuple, $u = \langle id, lp, rp, vec \rangle$, where id is an identity, vec is an m -dimensional vector, and lp and rp point to the left and right child nodes of u , respectively. The setting of u depends on the position of u in $\mathcal{I}$ as follows:

- if u is a leaf node, then u corresponds to a document which is assumed to be f_i , and the setting of u is $u.lp = u.rp = \emptyset$, $u.id = i$ and $u.vec = \Gamma[i]$.
- if u is not a leaf node, then $u.lp$ and $u.rp$ point to the left and right child nodes of u , $u.id = \emptyset$, and $u.vec$ is the maximized filter vector of $u.lp$ and $u.rp$ where $u.vec = MFV(\{u.lp.vec, u.rp.vec\})$.

On the basis of Definition 2, we give the construction algorithm of the encrypted index, *BuildIndex*, which is performed by the data owner.

$\hat{\mathcal{I}} \leftarrow \mathbf{BuildIndex}(\Gamma, K)$. The data owner first takes the document topic vectors of $\Gamma = \{\Gamma[1], \Gamma[2], \dots, \Gamma[n]\}$ as the input and uses the divisive hierarchical clustering algorithm to construct the plaintext BCI-tree index. Then, the data owner encrypts the index with the private keys of K . The details of the *BuildIndex* algorithm is shown as follows:

- 1) Taking the document topic vectors of Γ as input, the data owner performs the *GenBCItree* algorithm to construct the plaintext BCI-tree index $\mathcal{I}$. The detail of *GenBCItree* is shown in Algorithm 1.
- 2) For each node in $\mathcal{I}$, the data owner uses the m -bit random vector S in K to split the vector V stored in the node into two vectors, V' and V'' . If $S[i] = 0$, then $V'[i] = V''[i] = V[i]$; and if $S[i] = 1$, then $V'[i] + V''[i] = V[i]$. Afterwards, the data owner uses the random invertible matrices M_1 and M_2 in K to encrypt V' and V'' , respectively, where $\hat{V} = \{\hat{V}', \hat{V}''\} = \{M_1^T V', M_2^T V''\}$. When the vectors of all nodes have been processed, the encrypted index $\hat{\mathcal{I}}$ is generated.

In the end of the index building module, the data owner uses the key k_f in K to encrypt the documents of F . And then, the encrypted documents are outsourced to the cloud server together with the encrypted index.

4.3 Multi-keyword Search Module

There are two key algorithms in the multi-keyword search module, *GenTrapdoor* and *MKSearch*. The former is to generate the trapdoor corresponding to a search with multiple queried keywords and the latter is to perform the search processing in privacy-preserving manner.

$\hat{V}_Q \leftarrow \mathbf{GenTrapdoor}(Q, \Psi, P_T, P_W, K)$. We assume that the started search is $Q = \{w_{e_1}, w_{e_2}, \dots, w_{e_p}\}$, where w_{e_p} is the e_p th keyword in D , and the search requests the k documents with the k highest semantic relevance scores to Q . The data user first generates the corresponding query topic vector V_Q and then uses the private keys of K to encrypt V_Q to get the trapdoor $\hat{V}_Q$. We describe in detail the generation process of trapdoor as follows:

Algorithm 1: *GenBCItree*(C)

Input: C is a cluster of document topic vectors representing a set of corresponding documents
Output: the root of a BCI-tree

- 1: Create a new BCI-tree node u ;
- 2: **if** $|C| > 1$ **then**
- 3: $u.id = \emptyset, u.vec = MFV(C)$;
- 4: Take the cluster C as the input to run the divisive hierarchical clustering algorithm, and the output sub-clusters are assumed to be C_1 and C_2 ;
- 5: $u.lp = GenBCItree(C_1)$;
- 6: $u.rp = GenBCItree(C_2)$;
- 7: **else if** $|C| = 1$ **then**
- 8: Assume that the only one document topic vector in C is $\Gamma[i]$ which corresponds to the document f_i ;
- 9: $u.vec = \Gamma[i], u.id = i, u.lp = u.rp = \emptyset$;
- 10: **end if**

- 1) The data user generates the query topic vector V_Q according to Eq.(3) where $\circ$ is the Hadamard product operator [29]. V_Q is an m -dimensional vector that represents the probabilities that the topic intent of Q belongs to each topic.

$$V_Q = \sum_{w_{e_i} \in Q} \frac{P_T \circ \Psi[e_i]}{|Q| \cdot P_W[e_i]} \quad (3)$$

- 2) The data user splits the vector V_Q into two random vectors $\{V'_Q, V''_Q\}$ with the n -bit random vector S in K . The split rule is the same as that in the second step of *BuildIndex*.
- 3) The data user encrypts $V_Q = \{V'_Q, V''_Q\}$ into $\hat{V}_Q = \{\hat{V}'_Q, \hat{V}''_Q\} = \{M_1^{-1}V'_Q, M_2^{-1}V''_Q\}$ by using the random inverted matrices M_1 and M_2 in K . The encrypted query topic vector $\hat{V}_Q$ is the trapdoor which is transmitted to the cloud server as the search command.

$R \leftarrow BCISearch(\hat{\mathcal{I}}, \hat{V}_Q, k, \tau)$. Once receiving the trapdoor $\hat{V}_Q$, the cloud server performs the *BCISearch* algorithm by using the encrypted index $\hat{\mathcal{I}}$ to obtain the search result R . The detail of *BCISearch* is shown in Algorithm 2.

According to Algorithm 2, *BCISearch* is a depth-first search algorithm. In the beginning of the algorithm, u is the root of the encrypted BCI-tree $\hat{\mathcal{I}}$ and $\tau = 0$. The pruning threshold τ can improve the search efficiency. For a sub-tree in $\hat{\mathcal{I}}$, if the inner product between the vector stored in the root of the subtree and the trapdoor vector is lower than τ , the subtree will be pruned because the inner product between the vector stored in any node of the subtree and the trapdoor vector must be lower than τ . And the documents stored in the leaf nodes of the subtree are definitely not in the search result. There is no need to check any nodes in the subtree.

In addition, the inner product between an encrypted document topic vector $\hat{I}[i] = \{M_1^T \Gamma'[i], M_2^T \Gamma''[i]\}$ and the encrypted query topic vector $\hat{V}_Q = \{M_1^{-1} V'_Q, M_2^{-1} V''_Q\}$ equals to the inner product between the corresponding plaintext vectors, i.e. $\hat{I}[i] \cdot \hat{V}_Q = \Gamma[i] \cdot V_Q$. The proof is shown in Eq.(4).

$$\begin{aligned}
\hat{I}[i] \cdot \hat{V}_Q &= \{V'_Q M_1^{-1}, V''_Q M_2^{-1}\} \cdot \{\Gamma'[i] M_1^T, \Gamma''[i] M_2^T\} \\
&= (V'_Q M_1^{-1}) \cdot (\Gamma'[i] M_1^T)^T + (V''_Q M_2^{-1}) \cdot (\Gamma''[i] M_2^T)^T \\
&= V'_Q \cdot \Gamma'[i] + V''_Q \cdot \Gamma''[i] \\
&= V_Q \cdot \Gamma[i].
\end{aligned} \tag{4}$$

It indicates that the cloud server can perform the multi-keyword search correctly even just knowing the encrypted data about documents and index, and the scheme proposed in this paper can achieve semantic-aware privacy-preserving multi-keyword search over encrypted cloud data.

Algorithm 2: $BCISearch(u, \hat{V}_Q, k, \tau)$

Input: u is the root of a subtree, $\hat{V}_Q$ is the trapdoor, k is the number of requested documents, and τ is the pruning threshold

Output: the search result R

```

1: if  $u \neq \emptyset$  then
2:   if  $u.vec \cdot \hat{V}_Q > \tau$  then
3:     if  $u$  is not a leaf node then
4:        $BCISearch(u.lp, \hat{V}_Q, k, \tau)$ ;
5:        $BCISearch(u.rp, \hat{V}_Q, k, \tau)$ ;
6:     else if  $u$  is a leaf node then
7:       Add  $u$  to  $R$ ;
8:       if  $|R| > k$  then
9:         Delete the document with the lowest score to  $\hat{V}_Q$  in  $R$ ;
10:      end if
11:     if  $|R| = k$  then
12:       Set  $\tau$  equal to the lowest value in the semantic relevance
       scores between each document in  $R$  and  $\hat{V}_Q$ ;
13:     end if
14:   end if
15: end if
16: end if

```

5 Performance Evaluation

In this section, we evaluate SAPMS and compare it with the schemes presented in [5, 30] which are denoted as TFIDF-MRSE and LDA-EMRSE, respectively.

We implement these schemes by using Python 3.6 in Windows 10 with Intel Core(TM) i5-9300H. The evaluation is based on two metrics, the semantic precision of search result and the search time cost. The dataset used in the evaluation is 20 newsgroups [14]. The default setting of the number of documents n , the number of topics m and the number of required documents k are $n = 8000$, $m = 100$ and $k = 100$, respectively.

5.1 Semantic Precision Evaluation

Assuming that the queried keywords come from the same category and the documents in different categories are semantically irrelevant, we evaluate the semantic precision of three schemes by using the calculation method adopted by [10], which is shown in Eq.(5),

$$Precision = \frac{TP}{TP + FP} \times 100\% \quad (5)$$

where TP and FP are the numbers of documents in the search result that belong and do not belong to the category of the search intent, respectively.

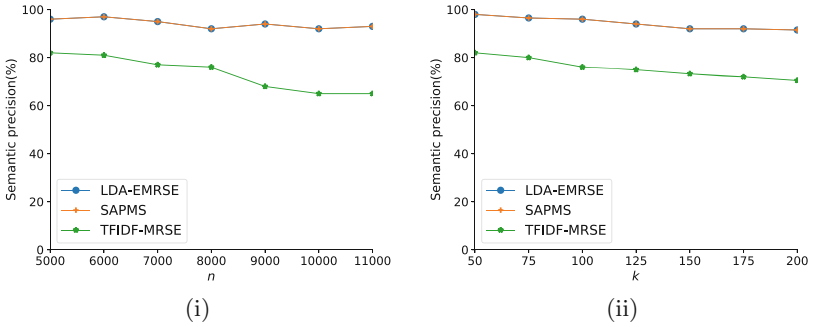


Fig. 2. Semantic precision versus n and k .

- 1) Fig. 2(i) shows that as the number of documents n increases, the semantic precision of SAPMS and LDA-EMRSE remains stable, while the semantic accuracy of TFIDF-MRSE gradually decreases. The reason is that in SAPMS and LDA-EMRSE, as the number of documents increases, the topics extracted by the LDA model can better reflect the semantic features of the documents but slightly affect the semantic precision of search results.
- 2) Fig. 2(ii) shows that as the number of documents k to be returned increases, the semantic precision of the three schemes decreases gradually. The reason is that as k increases, documents with lower relevance scores to the search intent are added to the search result, which lead to a slight decrease in semantic precision.

In addition, Fig. 2(i) and 2(ii) both show that the semantic precision of SAPMS and LDA-EMRSE are the same and higher than that of TFIDF-MRSE. The reason is that the former two schemes are based on the LDA model and have the ability to find the same result, thus they have the same semantic precision.

5.2 Search Time Cost Evaluation

Figure 3(i) shows that as the number of documents n increases, the search time cost of all three schemes increases. The reason is that three schemes all need to traverse the nodes of the index trees and perform the inner product to obtain the search result. The larger n is, the more nodes need to be traversed, so the inner product increases, which leads to the increase of search time cost.

Figure 3(ii) shows that as the number of requested documents k increases, the search time cost of all three schemes increases. The reason is that with the increase of k , the search processing of three schemes all need to traverse more nodes. It means that more inner products need to be calculated, which increases the search time cost.

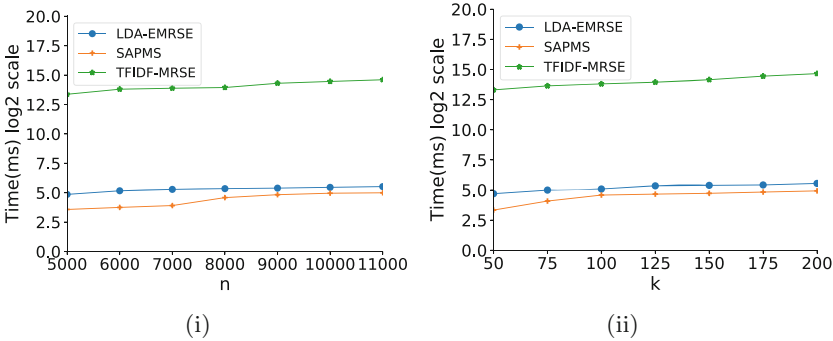


Fig. 3. Time Cost versus n and k .

The experimental results in Fig. 3(i) and 3(ii) both show that the search time cost of TFIDF-MRSE is much higher than the other two schemes. The reason is that the dimension of the vector in TFIDF-MRSE is the same as the scale of the dictionary, which is much higher than the dimension of vectors in SAPMS and LDA-EMRSE. The search time cost of an inner product between vectors with small dimension is obviously faster than that with high dimension. Meanwhile, Fig. 3(i) and Fig. 3(ii) both show that the search time cost of SAPMS is less than that of LDA-EMRSE. The reason is that documents are randomly distributed in leaf nodes in LDA-EMRSE, while documents in SAPMS are well organized in the leaf nodes and documents with high relevance are usually near with each other. Thus, the search efficiency is improved.

6 Conclusion

The searchable encryption in the cloud environment is an effective method to provide multi-keyword searches while protect the privacy of the outsourced data. A semantic-aware privacy-preserving multi-keyword search scheme is proposed in this paper. By using the LDA model and the divisive hierarchical clustering algorithm, an novel tree-based index is presented, which supports semantic-aware multi-keyword searches over encrypted cloud data. The experimental results show that the proposed scheme has better performance in terms of semantic precision of search results and search time cost.

Acknowledgements. This work was supported by the National Natural Science Foundation of China under the grant Nos.61872197,61902199 and 61972209; the Jiangsu Province Postgraduate Scientific Research Innovation Program under Grand No.KYCX22.0984; the “14th Five-Year Plan” Civil Aerospace Pre-research Project of China (D020101).

References

1. Blei, D.M., Ng, A.Y., Jordan, M.I.: Latent dirichlet allocation. *J. Mach. Lear. Res.* **3**, pp. 993–1022 (2003)
2. Cao, N., Wang, C., Li, M., Ren, K., Lou, W.: Privacy-preserving multi-keyword ranked search over encrypted cloud data. *IEEE Trans. Parallel Distrib. Syst.* **1**(25), 222–233 (2014)
3. Chang, Y.-C., Mitzenmacher, M.: Privacy Preserving Keyword Searches on Remote Encrypted Data. In: Ioannidis, J., Keromytis, A., Yung, M. (eds.) *ACNS 2005*. LNCS, vol. 3531, pp. 442–455. Springer, Heidelberg (2005). https://doi.org/10.1007/11496137_30
4. Chen, C., et al.: An efficient privacy-preserving ranked keyword search method. *IEEE Trans. Parallel Distrib. Syst.* **27**(4), 951–963 (2016)
5. Dai, H., Dai, X., Yi, X., Yang, G., Huang, H.: Semantic-aware multi-keyword ranked search scheme over encrypted cloud data. *J. Network Comput. Appl.* **147**, 102442 (2019)
6. Dai, H., Yang, M., Yang, G., Xiang, Y., Hu, Z., Wang, H.: A keyword-grouping inverted index based multi-keyword ranked search scheme over encrypted cloud data. *IEEE Trans. Sustain. Comput.* **7**(3), pp. 561–578 (2022)
7. Dai, X., Dai, H., Rong, C., Yang, G., Xiao, F.: Enhanced semantic-aware multi-keyword ranked search scheme over encrypted cloud data. *IEEE Trans. Cloud Comput.* **10**(4), 2595–2612 (2022)
8. Dai, Y., Shao, J., Hu, G., Guo, L.: A resource-aware approach for authenticating privacy preserving GNN queries. *World Wide Web* **22**(2), 437–454 (2019)
9. Fu, Z., Sun, X., Ji, S., Xie, G.: Towards efficient content-aware search over encrypted outsourced data in cloud. In: *IEEE INFOCOM 2016-The 35th Annual IEEE International Conference on Computer Communications*, pp. 1–9 IEEE (2016)

10. Gabryel, M., Damaševičius, R., Przybyszewski, K.: Application of the Bag-of-Words Algorithm in Classification the Quality of Sales Leads. In: Rutkowski, L., Scherer, R., Korytkowski, M., Pedrycz, W., Tadeusiewicz, R., Zurada, J.M. (eds.) ICAISC 2018. LNCS (LNAI), vol. 10841, pp. 615–622. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-91253-0_57
11. Hozhabr, M., Asghari, P., Javadi, H.: Dynamic secure multi-keyword ranked search over encrypted cloud data. *J. Inf. Secur. Appl.* **61**(1), 102902 (2021)
12. Ibrahim, A., Jin, H., Yassin, A.A., Zou, D.: Secure rank-ordered search of multi-keyword trapdoor over encrypted cloud data. In: 2012 IEEE Asia-Pacific Services Computing Conference, pp. 263–270. IEEE (2012)
13. Kiayias, A., Oksuz, O., Russell, A., Tang, Q., Wang, B.: Efficient Encrypted Keyword Search for Multi-user Data Sharing. In: Askoxylakis, I., Ioannidis, S., Katsikas, S., Meadows, C. (eds.) ESORICS 2016. LNCS, vol. 9878, pp. 173–195. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-45744-4_9
14. Lang, K.: NewsWeeder: Learning to filter Netnews. In: Machine Learning Proceedings 1995. Elsevier, pp. 331–339. (1995)
15. Li, B., Liang, R., Zhou, W., Yin, H., Gao, H., Cai, K.: LBS meets blockchain: An efficient method with security preserving trust in SAGIN. *IEEE Internet Things Journal* **9**(8), 5932–5942 (2022)
16. Li, B., Liang, R., Zhu, D., Chen, W., Lin, Q.: Blockchain-based trust management model for location privacy preserving in VANET. *IEEE Trans. Intell. Transp. Syst.* **22**(6), 3765–3775 (2021)
17. Li, J., Wang, Q., Wang, C., Cao, N., Ren, K., Lou, W.: Fuzzy keyword search over encrypted data in cloud computing. In: 2010 Proceedings IEEE INFOCOM, pp. 1–5. IEEE (2010)
18. Liang, Y., Li, Y., Zhang, K., Ma, L.: DMSE: dynamic multi-keyword search encryption based on inverted index. *J. Syst. Archit.* **119**, 102255 (2021)
19. Liu, C., Zhu, L., Chen, J.: *J. Network Comput. Appl.* **86**, 3–14 (2017)
20. Liu, Q., Peng, Y., Wu, J., Wang, T., Wang, G.: Secure multi-keyword fuzzy searches with enhanced service quality in cloud computing. *IEEE Trans. Netw. Serv. Manage.* **18**(2), 2046–2062 (2020)
21. Orencik, C., Kantarcioglu, M., Savas, E.: A practical and secure multi-keyword search method over encrypted cloud data. In: 2013 IEEE Sixth International Conference on Cloud Computing, pp. 390–397. IEEE (2013)
22. Padhya, M., Jinwala, D.C.: R-OO-KASE: revocable online offline key aggregate searchable encryption. *Data Sci. Eng.* **5**(4), 391–418 (2020)
23. Roux, M.: A comparative study of divisive and agglomerative hierarchical clustering algorithms. *J. Classif.* **35**(2), 345–366 (2018)
24. Swaminathan, A., et al.: Confidentiality-preserving rank-ordered search. In: Proceedings of the 2007 ACM workshop on Storage security and survivability, pp. 7–12 (2007)
25. Tseng, C.Y., Lu, C., Chou, C.F.: Efficient privacy-preserving multi-keyword ranked search utilizing document replication and partition. In: 2015 12th Annual IEEE Consumer Communications and Networking Conference (CCNC), pp. 671–676. IEEE (2015)
26. Wang, B., Yu, S., Lou, W., Hou, Y.T.: Privacy-preserving multi-keyword fuzzy search over encrypted data in the cloud. In: IEEE INFOCOM 2014-IEEE Conference on Computer Communications, pp. 2112–2120. IEEE (2014)
27. Wang, C., Cao, N., Li, J., Ren, K., Lou, W.: Secure ranked keyword search over encrypted cloud data. In: 2010 IEEE 30th International Conference on Distributed Computing Systems, pp. 253–262. IEEE (2010)

28. Wang, C., Cao, N., Ren, K., Lou, W.: Enabling secure and efficient ranked keyword search over outsourced cloud data. *IEEE Trans. Parallel Distrib. Syst.* **23**(8), 1467–1479 (2011)
29. Wang, P., Ravishankar, C.V.: On masking topical intent in keyword search. In: 2014 IEEE 30th International Conference on Data Engineering, pp. 256–267. IEEE (2014)
30. Xia, Z., Wang, X., Sun, X., Wang, Q.: A secure and dynamic multi-keyword ranked search scheme over encrypted cloud data. *IEEE Trans. Parallel Distrib. Syst.* **27**(2), 340–352 (2015)
31. Xiangyang, Z., Hua, D., Xun, Y., Geng, Y., Xiao, L.: Muse: an efficient and accurate verifiable privacy-preserving multikeyword text search over encrypted cloud data. *Secur. Commun. Netw.* **2017**, 1–17 (2017)
32. Zerr, S., Olmedilla, D., Nejdl, W., Siberski, W.: Zerber+r: top-k retrieval from a confidential index. In: *Proceedings of the 12th International Conference on Extending Database Technology: Advances in Database Technology*, pp. 439–449 (2009)
33. Zhou, Q., Dai, H., Shen, W., Liu, Y., Yang, G.: Evss: An efficient verifiable search scheme over encrypted cloud data. *World Wide Web* pp. 1–21 (2022)